INTRODUCTION TO PYTHON
UNIT 2: CONTROL FLOW, LOOPS, MODULAR FUNCTIONS & RECURSION
Curriculum: Lovely Professional University | School of Electronics & Communication Engineering
Reference: Python Programming: Using Problem Solving Approach by Reema Thareja

>> 2.1 Decision Making and Conditional Control Structures

Conditional constructs direct program execution flow along branching paths based on boolean evaluations:
* 1. Single if Statement: Evaluates test expression; executes indented block if condition evaluates to True.
* 2. if-else Statement: Provides binary alternative branch (executes else block when test evaluates to False).
* 3. if-elif-else Ladder: Evaluates multi-way conditions sequentially from top to bottom. Once a condition evaluates to True, its block executes and the remaining ladder is completely bypassed.
* 4. Nested Conditionals: if statements placed inside other if or else blocks. Code readability maintained via consistent 4-space indentation.
* Short-Circuit Evaluation: In logical expressions A and B, if A is False, B is never evaluated. In A or B, if A is True, B is never evaluated.

>> 2.2 Iterative Control: while and for Loops

* 1. while Loop: Pre-test loop executing repeatedly as long as condition remains True. Mandates loop variable initialization, condition check, and loop variable update to prevent infinite loops.
* 2. for Loop: Traverses over elements of any iterable sequence (string, list, tuple, range, dictionary).
* range() Built-in: range(start, stop, step) generates an immutable arithmetic sequence. Omission defaults: start=0, step=1.
* Loop Control Statements:
- break: Terminates execution of innermost loop immediately, transferring control to statement following loop.
- continue: Skips remainder of current iteration and jumps immediately to next iteration cycle.
- pass: Null statement acting as syntactic placeholder where code is syntactically required but no action is desired.
* Loop else Clause: Unique Python feature. The else block executes when loop finishes naturally without encountering a break statement.

>> 2.3 Modular Programming and Function Definitions

* Function Syntax: def function_name(param1, param2=default): followed by docstring, body, and return statement.
* Parameter Passing Semantics: Pass-by-Object-Reference (Call-by-Assignment):
- Immutable arguments (int, float, str, tuple): Behave like call-by-value; caller variable remains unmodified.
- Mutable arguments (list, dict, set): Modified in-place inside function; changes reflect in caller.
* Argument Classification:
- Positional Arguments: Mapped in order of appearance.
- Keyword Arguments: Explicitly specified by name (e.g. func(b=2, a=1)), order independent.
- Default Arguments: Assigned default value in header. Warning: Never use mutable objects (like []) as default arguments!
- Variable-Length Arguments: *args collects extra positional arguments into a tuple; **kwargs collects extra keyword arguments into a dictionary.

>> 2.4 Scope Rules and Lambda Functions

* The LEGB Scope Resolution Rule: Local -> Enclosing -> Global -> Built-in.
* global keyword: Allows modifying a module-level global variable from within a local function scope.
* nonlocal keyword: Allows modifying a variable in an outer enclosing non-global scope (closures).
* Lambda Functions: Compact anonymous single-expression functions: lambda x, y: x + y.
* Functional Primitives: map(func, iter), filter(func, iter), and functools.reduce(func, iter).

>> 2.5 Recursion: Mechanics and Stack Frames

* Recursion: A programming technique where a function calls itself directly or indirectly to solve smaller sub-problems.
* Essential Components: Base Case (terminating condition preventing infinite recursion) and Recursive Step (reducing problem size).
* Call Stack: Every recursive call allocates a new activation record (stack frame) on the runtime call stack. Python enforces a safety recursion limit (default 1000, inspectable via sys.getrecursionlimit()).

[PROGRAM 1: Resistor Color Code Decoder and Tolerance Verification]

Problem Context: Decode standard 4-band resistor color codes, calculate nominal resistance and tolerance bounds, and verify whether a measured resistance falls within

specification.
Python Source Code:

```
# Standard EIA color code lookup tables
COLOR_DIGITS = {'black': 0, 'brown': 1, 'red': 2, 'orange': 3, 'yellow': 4,
'green': 5, 'blue': 6, 'violet': 7, 'gray': 8, 'white': 9}

COLOR_MULTIPLIER = {'black': 1, 'brown': 10, 'red': 100, 'orange': 1000,
'yellow': 10000, 'green': 100000, 'blue': 1000000,
'gold': 0.1, 'silver': 0.01}

COLOR_TOLERANCE = {'brown': 1.0, 'red': 2.0, 'gold': 5.0, 'silver': 10.0, 'none': 20.0}

def decode_resistor(b1, b2, b3, b4='gold'):
b1, b2, b3, b4 = b1.lower(), b2.lower(), b3.lower(), b4.lower()
if b1 not in COLOR_DIGITS or b2 not in COLOR_DIGITS:
return 'Invalid band 1 or 2 color!'
if b3 not in COLOR_MULTIPLIER or b4 not in COLOR_TOLERANCE:
return 'Invalid multiplier or tolerance color!'

significant_digits = (COLOR_DIGITS[b1] * 10) + COLOR_DIGITS[b2]
nominal_resistance = significant_digits * COLOR_MULTIPLIER[b3]
tolerance_pct = COLOR_TOLERANCE[b4]

lower_limit = nominal_resistance * (1 - tolerance_pct / 100)
upper_limit = nominal_resistance * (1 + tolerance_pct / 100)

return nominal_resistance, tolerance_pct, lower_limit, upper_limit

# Example: Yellow - Violet - Orange - Gold (47 kOhm +/- 5%)
nom, tol, low, high = decode_resistor('yellow', 'violet', 'orange', 'gold')
print(f'Nominal Resistance : {nom} Ohms ({nom/1000:.1f} kOhms)')
print(f'Tolerance : +/- {tol} %')
print(f'Acceptable Range : {low:.1f} Ohms to {high:.1f} Ohms')
```

[PROGRAM 2: Prime Number Generator & Factorizer using Loop-Else]

Problem Context: Generate all prime numbers in an engineering range and decompose composite numbers into prime factors using for-else loops.
Python Source Code:

```
def find_primes_in_range(start, end):
primes = []
for num in range(start, end + 1):
if num < 2:
continue
```

```
for i in range(2, int(num**0.5) + 1):
if num % i == 0:
break
else:
# Executes only if loop completed without break
primes.append(num)
return primes

def prime_factorization(n):
factors = []
d = 2
while d * d <= n:
while n % d == 0:
factors.append(d)
n //= d
d += 1
if n > 1:
factors.append(n)
return factors

print('Primes between 20 and 60:', find_primes_in_range(20, 60))
print('Prime factors of 840:', prime_factorization(840))
```

[PROGRAM 3: Taylor Series Approximation of Sine using Recursive Factorial]

Problem Context: Compute sin(x) using Taylor series expansion up to N terms and compare accuracy against math.sin().

Python Source Code:

```
import math

def factorial_rec(n):
if n <= 1:
return 1
return n * factorial_rec(n - 1)

def taylor_sin(x_rad, terms=7):
# sin(x) = sum_{k=0}^{terms-1} (-1)^k * x^(2k+1) / (2k+1)!
sin_approx = 0.0
for k in range(terms):
numerator = ((-1)**k) * (x_rad**(2 * k + 1))
denominator = factorial_rec(2 * k + 1)
sin_approx += numerator / denominator
return sin_approx
```

```
angle_deg = 45.0
x = math.radians(angle_deg)
approx = taylor_sin(x, terms=6)
exact = math.sin(x)
print(f'Angle: {angle_deg} degrees ({x:.4f} rad)')
print(f'Taylor Series Approximation : {approx:.10f}')
print(f'math.sin() Library Value : {exact:.10f}')
print(f'Absolute Error : {abs(approx - exact):.2e}')
```

[PROGRAM 4: Functional Sensor Calibration Pipeline using Map & Filter]

Problem Context: Filter out out-of-bounds electrical noise from an ADC reading stream and calibrate raw integers into physical temperature readings using functional programming.

Python Source Code:

```
raw_adc_readings = [12, 105, 340, 1025, 512, -4, 780, 2048, 420]

# 1. Filter valid 10-bit ADC range [0 to 1023]
valid_readings = list(filter(lambda x: 0 <= x <= 1023, raw_adc_readings))

# 2. Map ADC reading to Voltage: V = (ADC / 1023) * 5.0 Volts
voltages = list(map(lambda adc: round((adc / 1023.0) * 5.0, 3), valid_readings))

# 3. Map Voltage to Temperature (LM35 sensor: 10 mV/C -> T = V / 0.010)
temperatures = list(map(lambda v: round(v / 0.010, 1), voltages))

print('Raw ADC Stream :', raw_adc_readings)
print('Filtered ADC Values :', valid_readings)
print('Calibrated Voltages :', voltages)
print('Final Temperatures C:', temperatures)
```